



Projet d'intégration d'un CNN

De l'algorithme à l'architecture
3A SoC

Auteurs : ONOKOMBA Ordan et DAOUDI Milane

2025

Table des matières

1	Modèle fonctionnel du CNN en Python	1
1.1	Rôle du modèle de référence	1
1.2	Chargement des coefficients du réseau	1
1.3	Prétraitement des images CIFAR-10	2
1.4	Implémentation manuelle des couches du CNN	2
1.5	Évaluation des performances :	3
2	Modèle virgule fixe et implémentation HLS	4
2.1	Prétraitement des images en HLS	4
2.2	Chargement et organisation des coefficients	5
2.3	Implémentation HLS des couches du CNN	5
2.3.1	Linéarisation des données	5
2.3.2	Convolution paramétrable	6
2.3.3	Couches non linéaires et classification	6
2.4	Fonction top et validation globale	7
3	Implémentation et validation matérielle	9
3.1	Images et coefficients en mémoire	10
3.2	Simulation RTL avec ModelSim	11
3.3	Génération du design et intégration Vivado	12
3.4	Validation matérielle	13
4	Conclusion générale	14
5	Annexes	16

1 Modèle fonctionnel du CNN en Python

1.1 Rôle du modèle de référence

La première étape du projet consiste à développer un modèle fonctionnel de référence du réseau de neurones convolutif (CNN) en Python. Ce modèle s'inscrit dans une démarche descendante visant à valider l'architecture, les calculs et les résultats du réseau avant toute transposition en virgule fixe puis en matériel.

Ce modèle de référence remplit trois rôles essentiels :

- valider la conformité fonctionnelle de l'architecture CNN spécifiée pour CIFAR-10,
- servir de référence numérique pour la validation des versions C++ et HLS,
- fournir un point de comparaison objectif pour l'évaluation des pertes de précision.

L'implémentation est volontairement réalisée sans bibliothèques de *deep learning* haut niveau afin de conserver une parfaite maîtrise des calculs et des dimensions intermédiaires.

1.2 Chargement des coefficients du réseau

Les poids et biais du réseau sont fournis sous forme d'un fichier texte (`CNN_coeff_3x3.txt`) issu d'un apprentissage préalable. Ils sont chargés par un module dédié, indépendant du reste du flot de calcul.

Cette séparation permet :

- une correspondance explicite entre les paramètres et les couches du réseau,
- un contrôle précis des dimensions des tenseurs,
- une réutilisation directe des coefficients lors de la transition vers le C++ et le HLS.

Les coefficients sont ainsi organisés en structures cohérentes avec la description théorique du CNN, facilitant la traçabilité entre modèle mathématique, implémentation logicielle et version matérielle.

1.3 Prétraitement des images CIFAR-10

Les images CIFAR-10 sont initialement fournies sous forme binaire (`data_batch_1.bin`), avec une résolution de 32×32 pixels et trois canaux RGB. Le prétraitement appliqué respecte strictement les contraintes du sujet et constitue une étape déterminante pour la qualité des résultats.

Il comprend :

- la reconstruction des images RGB à partir du flux binaire,
- un recadrage centré vers une résolution 24×24 ,
- une normalisation globale des pixels par la moyenne et l'écart-type.

La normalisation est réalisée sur l'ensemble des canaux afin de conserver les relations chromatiques, particulièrement importantes pour la reconnaissance d'objets naturels.

1.4 Implémentation manuelle des couches du CNN

L'ensemble des couches du CNN a été implémenté manuellement, sans abstraction logicielle externe. Ce choix permet une correspondance directe entre l'implémentation et les équations mathématiques du réseau, tout en préparant la future transposition matérielle.

Convolution multi-canaux :

La convolution 2D est implémentée par balayage spatial explicite, avec accumulation des produits pondérés sur l'ensemble des canaux d'entrée. Le padding de type same est utilisé afin de conserver les dimensions spatiales des feature maps, ce qui simplifie l'enchaînement des couches et le suivi des dimensions.

Fonctions non linéaires et sous-échantillonnage :

La fonction ReLU est appliquée élément par élément pour introduire la non-linéarité. Le MaxPooling permet de réduire la résolution spatiale tout en conservant les activations dominantes, contribuant à la robustesse du réseau face aux variations locales.

Classification :

La couche entièrement connectée est implémentée sous forme d'un produit matrice-vecteur suivi de l'ajout du biais. Une fonction Softmax, avec stabilisation numérique, est appliquée afin d'obtenir une distribution de probabilités

exploitable pour l'évaluation des performances.

1.5 Évaluation des performances :

Le CNN fonctionnel a été évalué sur un sous-ensemble de 3 000 images issues de la base CIFAR-10.

```
=====
Global accuracy on 3000 images = 63.53%
=====

Confusion matrix:
[[144  17  14   3   8   2   5   1  91  14]
 [   2 231   4   2   4   2   4   0  15  23]
 [  15   3 198  16  12   2  50   6  17   3]
 [   2   6  24 144  32  13  45   7   7   5]
 [   7   3  27   9 188   1  49  14  11   2]
 [   4   6  15  58  25 115  23  21   7   5]
 [   3   3  35  10   8   4 237   4   5   3]
 [   7   8  14  14  51   3  20 162  11   7]
 [  12   5   4   5   2   0   5   1 266   8]
 [   4  25   7   4   5   0   4   2  28 221]]
```

FIGURE 1 – Performances du CNN – modèle Python

Le taux de reconnaissance obtenu est :

$$\text{Accuracy} = 63,53\%$$

Ce résultat est cohérent avec la profondeur modérée du réseau et confirme la validité fonctionnelle de l'implémentation. Le modèle Python constitue ainsi une référence fiable pour l'analyse des écarts numériques introduits lors du passage en virgule fixe et de l'implémentation HLS.

Les confusions observées concernent principalement des classes visuellement proches (chat/chien, oiseau/avion), mettant en évidence les limites d'un CNN peu profond sur des images de faible résolution. Ces résultats serviront de base de comparaison directe pour les phases matérielles du projet.

2 Modèle virgule fixe et implémentation HLS

Après validation du modèle fonctionnel en Python, la seconde phase du projet consiste à transposer progressivement le CNN vers une implémentation en virgule fixe compatible avec une synthèse matérielle. Cette étape intermédiaire repose sur l'outil *Catapult C* et vise à préparer une future intégration FPGA tout en conservant une référence fonctionnelle vérifiable.

Le flot de traitement est structuré autour de quatre briques principales :

- `preprocess` : préparation des données d'entrée (image),
- `load_coeff` : chargement et organisation des coefficients,
- `cnn_hls` : implémentation des couches du réseau,
- `forward_debug` : intégration globale du flot d'inférence.

Chaque module est validé indépendamment avant intégration, afin de maîtriser la complexité du passage vers le matériel.

Choix de la représentation numérique :

Les calculs sont réalisés en virgule fixe à l'aide du type `ac_fixed`. Le format retenu n'est pas optimisé à ce stade, mais permet d'assurer une précision suffisante tout en restant synthétisable.

Listing 1 – Types utilisés

```
1 typedef ac_fixed<PIXEL_OUT_WIDTH, 9, true, AC_RND_INF, AC_SAT> pixel_out_t;  
2 typedef ac_fixed<35, 16, true, AC_RND_INF, AC_SAT> acc_t;
```

Les produits de convolution sont accumulés dans un type intermédiaire de largeur étendue afin de limiter les erreurs de quantification et d'éviter les débordements lors des sommes de produits. Ce choix facilite l'analyse ultérieure de l'impact de la quantification sur la précision globale du réseau.

2.1 Prétraitement des images en HLS

Le module de prétraitement constitue l'interface entre les données d'entrée et le CNN. Il réorganise les pixels dans un format linéaire compatible avec le flot HLS et garantit des valeurs normalisées dans l'intervalle attendu.

```
1 void preprocess (  
2     ac_fixed<PIXEL_out_WIDTH,8,true,AC_RND_INF,AC_SAT > image_buffer[IMG_SIZE  
3     ],  
4     ac_fixed<PIXEL_out_WIDTH,8,true,AC_RND_INF,AC_SAT > img_out[IMG_CROP_SIZE]  
5 );
```

Contrairement à la version Python, la normalisation complète par la moyenne et l'écart-type n'est pas intégrée directement dans ce module, conformément aux recommandations du sujet. Cette séparation permet de simplifier le flot matériel tout en conservant une référence logicielle claire.

La validation est réalisée via un banc de test compatible **SCVerify**, comparant systématiquement les sorties du module HLS avec celles du modèle Python. La seule chose qui changera lors du test sur carte sera la façon dont les images sont données à la fonction **preprocess**. Ici c'est le testbench qui lui apporte les images. Sur carte, elles seront directement en mémoire RAM.

2.2 Chargement et organisation des coefficients

Avant toute synthèse matérielle, un modèle C++ intermédiaire est utilisé pour valider le chargement et l'organisation des coefficients du réseau. Les poids et biais sont extraits depuis un fichier texte et stockés dans une structure associative reliant explicitement chaque couche à ses paramètres.

Les coefficients sont ensuite copiés dans des tableaux statiques de dimensions fixes, strictement conformes à l'architecture du CNN. Cette étape permet de garantir la cohérence des paramètres avant suppression complète des accès fichiers lors de l'intégration matérielle. C'est pour ça qu'on ne détaille pas cette partie qui nous permettra seulement de tester le CNN avec fonctions en HLS (sous-section 2.4).

2.3 Implémentation HLS des couches du CNN

2.3.1 Linéarisation des données

Les tenseurs multidimensionnels utilisés en Python sont remplacés par des tableaux unidimensionnels. Les données sont stockées selon un ordre mémoire explicite (row-major, HWC), et des fonctions d'indexation dédiées assurent la conversion entre indices logiques et adresses linéaires.

Listing 2 – Fonctions d'indexation

```

1 inline int IDX3(int h, int w, int c, int W, int Cin) {
2     return (h * W * Cin) + (w * Cin) + c;
3 }
4 inline int IDX4(int kh, int kw, int ci, int co, int K, int Cin, int Cout) {
5     return (kh * K * Cin * Cout) + (kw * Cin * Cout) + (ci * Cout) + co;
6 }

```

Cette approche rend le mapping mémoire entièrement explicite et parfaitement compatible avec la synthèse matérielle.

2.3.2 Convolution paramétrable

La convolution 2D est implémentée sous forme de fonction `template`, avec des dimensions entièrement connues à la compilation.

Listing 3 – Prototype de convolution HLS

```

1 template<int H, int W, int Cin, int K, int Cout, int STRIDE, bool SAME>
2 void conv2d_hls(
3     const pixel_out_t x[H*W*Cin],
4     const fixed_t Wf[K*K*Cin*Cout],
5     const fixed_t b[Cout],
6     pixel_out_t out[((SAME ? H : (H-K+1))/STRIDE) *
7                     ((SAME ? W : (W-K+1))/STRIDE) * Cout]
8 );

```

Les boucles de calcul sont statiques, ce qui facilite l'analyse du flot de données et prépare l'introduction ultérieure d'optimisations HLS (pipeline, déroulement).

2.3.3 Couches non linéaires et classification

Les couches ReLU, MaxPooling, flatten et entièrement connectées sont transposées directement depuis le modèle Python. Leur implémentation repose exclusivement sur des opérations simples (comparaisons, accumulations), favorables à une synthèse efficace.

Listing 4 – Couche entièrement connectée

```

1 template<int N, int M>
2 void dense_hls(
3     const pixel_out_t x[N],
4     const fixed_t W[N*M],
5     const fixed_t b[M],
6     pixel_out_t out[M]
7 ){
8     for(int j=0;j<M;j++){
9         acc_t s = b[j];
10        for(int i=0;i<N;i++)
11            s += x[i] * W[i*M + j];
12        out[j] = s;
13    }
14 }

```


2.4 Fonction top et validation globale

Une fonction `top` regroupe l'ensemble du flot d'inférence du CNN et constitue le point d'entrée de la synthèse HLS. Elle initialise des tableaux 1D pour les résultats intermédiaires et appelle les différentes fonctions du CNN.

Listing 5 – Fonction top HLS

```
1 void forward_debug_hls(  
2     const int img_sel,  
3     int &pred_out,  
4     bool &pred_valid  
5 ){  
6     // Par défaut : pas valide  
7     pred_valid = false;  
8     // Processing couches du CNN ...  
9 }
```

Un banc de test logiciel permet de comparer systématiquement les sorties HLS avec celles du modèle Python, garantissant la continuité fonctionnelle entre les différentes versions du CNN.

```
Image 2994: True label = 1, Predicted label = 8  
Image 2995: True label = 4, Predicted label = 2  
Image 2996: True label = 0, Predicted label = 0  
Image 2997: True label = 6, Predicted label = 6  
Image 2998: True label = 6, Predicted label = 2  
Image 2999: True label = 4, Predicted label = 4  
  
=====  
Accuracy on 3000 images = 63.5333%  
=====
```

FIGURE 2 – Résultats avec Catapult du CNN HLS sur 3000 images

On observe une précision équivalente à celle du CNN en Python (résultat sur 10 images voir 13). Cela est certainement dû au fait qu'on utilise une virgule fixe avec beaucoup de bits.

Conclusion intermédiaire

À l'issue de cette phase, le projet dispose désormais d'un ensemble cohérent de fonctions HLS telles que `preprocess`, `conv2d`, `maxpool`, `relu`, `dense` et `forward_debug`, dont le comportement fonctionnel est validé et compatible avec une synthèse matérielle. Ces briques reposent exclusivement sur des opérations élémentaires (additions, multiplications, comparaisons), des boucles

à bornes statiques et des tableaux unidimensionnels, ce qui les rend directement synthétisables et particulièrement adaptées à une optimisation HLS.

À ce stade, l'architecture n'est pas encore optimisée en termes de performances. Des améliorations significatives pourraient être apportées par l'introduction de techniques de partitionnement mémoire, permettant le déroulement de boucles internes. En effet, les mémoires embarquées sur FPGA disposent généralement d'un nombre limité de ports (un port de lecture et un port d'écriture), ce qui empêche le parallélisme tant que les données intermédiaires ne sont pas correctement partitionnées.

Résultats de performance affichés par Catapult : Nous avons juste eu le temps de pipeliner certaines opérations et avons pu observer une diminution du temps de calcul par image d'un facteur 2 et une légère réduction de la surface. Il aurait été intéressant de pouvoir comparer la surface et le temps de calcul en fonction d'un plus grand niveau d'optimisation pour observer de réels changements.

Solution /	Latency Cycles	Latency Time	Throughput Cycles	Throughput Time	Slack	Total Area
 solution.v1 (new)						
 forward_debug_hls.v1 (extract)	8963971	179279420.00	8963972	179279440.00	0.31	10028.93
 forward_debug_hls.v2 (allocate)	4736105	94722100.00	4736106	94722120.00		8556.42

FIGURE 3 – Améliorations avec légères optimisations

Mémoire nécessaire sur carte FPGA en théorie : En l'état actuel, on calcule pour notre code une mémoire nécessaire sur FPGA de 293 kB ce qui est bien inférieur à la mémoire totale nécessaire.

Catégorie mémoire	Taille (bits)	Taille (KB)
Image d'entrée (24×24×3)	27 648	3.38
Paramètres du réseau (poids + biais)	445 536	54.4
Cartes de caractéristiques (buffers persistants)	846 880	103.4
Buffers de padding (convolutions + pooling, pire cas)	1 078 976	131.7
Total mémoire embarquée requise	2 399 040	293.0

Nombres de cycles théorique pour le calcul d'une image : Théoriquement, on trouve un temps de 8 M de cycles pour le calcul d'une image, alors que Catapult a calculé un nombre de cycles total de 9 M sans pipelining.

Bloc de calcul	Modèle de calcul	Cycles théoriques
Convolution 1	$24 \times 24 \times 64 \times 3 \times 3 \times 3$ MAC (mul+add = 2 cycles)	1 990 656
ReLU 1	$24 \times 24 \times 64$ comparaisons	36 864
MaxPool 1	$H_p W_p C + H W C + H_{out} W_{out} C \cdot 9$	163 072
Convolution 2	$12 \times 12 \times 32 \times 3 \times 3 \times 64$ MAC (mul+add = 2 cycles)	5 308 416
ReLU 2	$12 \times 12 \times 32$ comparaisons	4 608
MaxPool 2	$H_p W_p C + H W C + H_{out} W_{out} C \cdot 9$	21 248
Convolution 3	$6 \times 6 \times 20 \times 3 \times 3 \times 32$ MAC (mul+add = 2 cycles)	414 720
ReLU 3	$6 \times 6 \times 20$ comparaisons	720
MaxPool 3	$H_p W_p C + H W C + H_{out} W_{out} C \cdot 9$	3 620
Flatten	180 copies	180
Dense (180→10)	180×10 MAC (mul+add = 2 cycles)	3 600
Argmax	9 comparaisons	9
Total par image		7 947 713

TABLE 1 – Estimation du nombre de cycles théoriques par image

Changements restants à effectuer :

Enfin, pour aboutir à une implémentation entièrement synthétisable et autonome, il est nécessaire de supprimer toute dépendance au banc de test logiciel. Celui-ci fournit actuellement les images d'entrée, participe à la normalisation et assure le chargement des coefficients. Ces éléments devront être intégrés directement dans l'architecture matérielle (via des mémoires initialisées) pour avoir un flot d'inférence totalement indépendant et compatible avec une exécution sur FPGA.

3 Implémentation et validation matérielle

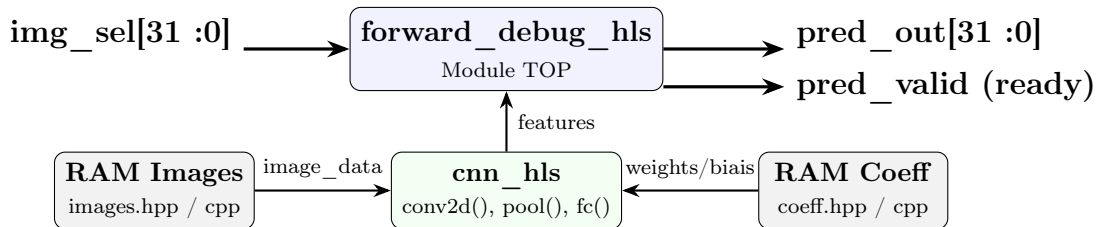


FIGURE 4 – Architecture HLS du module forward_debug_hls

Après la validation fonctionnelle du CNN en HLS, l'étape suivante consiste à produire une implémentation complète sur FPGA. Cette phase comprend

trois aspects principaux :

- l'intégration des données (images et coefficients) directement en mémoire matérielle,
- la vérification RTL du design généré par Catapult,
- la validation matérielle et le contrôle du bon fonctionnement via des signaux de supervision.

Le signal **pred_out** représente la classe prédite par le CNN, codée sur 32 bits, dont la valeur utile est comprise entre 0 et 9. Le signal **pred_valid** joue un rôle crucial : il indique explicitement la fin du calcul matériel et garantit que la valeur de **pred_out** est stable et exploitable. Ce mécanisme permet d'éviter toute lecture prématurée du résultat, en particulier dans un contexte matériel fortement séquentiel.

3.1 Images et coefficients en mémoire

Afin d'assurer un fonctionnement autonome du CNN sur FPGA, les images de test ainsi que les coefficients du réseau sont intégrés directement dans le design sous forme de mémoires ROM. Ils sont définis via des tableaux constants dans des fichiers d'en-tête (.hpp) et initialisés dans les fichiers source (.cpp) utilisés par Catapult.

Cette approche présente plusieurs avantages :

- suppression de toute dépendance à des fichiers externes ou à un banc de test logiciel,
- garantie d'une synthétisabilité complète du design,
- reproductibilité stricte entre les simulations HLS, RTL et l'exécution sur FPGA.

Listing 6 – Exemple d'initialisation des tableaux pour les coefficients et les images

```
1 /* Dans coeff.hpp */
2 extern const fixed_t W1[W1_SIZE];
3 /* Dans coeff.cpp */
4 const fixed_t W1[W1_SIZE] = { /* valeurs issues du modele Python */ };
```

Les valeurs numériques proviennent directement du modèle entraîné en Python, garantissant une continuité stricte entre l'entraînement logiciel et l'implémentation matérielle.

3.2 Simulation RTL avec ModelSim

La fonction top HLS `forward_debug_hls` est synthétisée en RTL (VHDL) à l'aide de Catapult C. Une simulation fonctionnelle RTL est ensuite effectuée à l'aide de ModelSim, lancée directement depuis l'environnement Catapult.

Cette étape permet :

- de vérifier que les sorties RTL correspondent aux résultats du modèle HLS,
- de détecter toute divergence entre le comportement algorithmique et le comportement matériel,
- d'assurer la continuité entre le modèle Python, le code HLS et le code RTL généré.

Afin de garantir un comportement matériel explicite et déterministe, les interfaces du module HLS sont configurées via le fichier `directive.tcl`. Les signaux d'entrée et de sortie sont forcés en interfaces directes (**DirectInput** / **DirectOutput**), ce qui empêche Catapult d'introduire des protocoles implicites ou des registres transactionnels non désirés.

```
1 directive set /forward_debug_hls/img_sel:rsc -MAP_TO_MODULE {[DirectInput]}
2 directive set /forward_debug_hls/pred_out:rsc -MAP_TO_MODULE {[DirectOutput
  ]}
3 directive set /forward_debug_hls/pred_valid:rsc -MAP_TO_MODULE {[
  DirectOutput]}
```

L'objectif de ces directives est d'imposer une interface matérielle simple et stable, dans laquelle :

- `img_sel` est continuellement accessible par le matériel,
- `pred_out` est directement drivé par le CNN,
- `pred_valid` signale explicitement la fin du calcul.

Cette configuration est indispensable pour une intégration fiable dans un top FPGA et pour une observation correcte des signaux lors des simulations RTL.

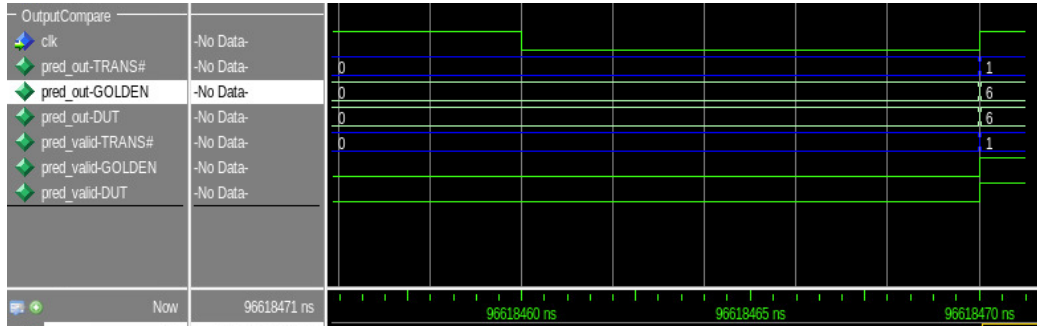


FIGURE 5 – Résultats de la simulation fonctionnelle RTL avec ModelSim

On observe la concordance complète entre les résultats attendus et les sorties matérielles. Les signaux `pred_valid_GOLDEN` et `pred_valid_DUT` passent tous deux à l'état haut à la fin du calcul.

De plus, les signaux `pred_out_GOLDEN` et `pred_out_DUT` sont identiques et égales à 6, soit la classe attendue pour l'image 0 par le modèle logiciel.

Il convient de noter que la simulation nécessite un temps relativement long (~ 96 ms de temps simulé), ce qui justifie le choix de limiter la validation à une seule image par exécution.

3.3 Génération du design et intégration Vivado

Catapult C génère, via Precision, le fichier `.edf` correspondant au CNN synthétisé. Ce fichier est ensuite intégré dans un projet Vivado via un wrapper VHDL (top), `forward.vhd`, chargé de connecter le coeur CNN aux entrées/sorties physiques de la carte FPGA.

La logique d'intégration comprend :

- l'instanciation du module HLS généré (`forward_debug_hls`) :

```

1 component forward_debug_hls
2   port (
3     clk      : in  std_logic;
4     rst      : in  std_logic;
5     img_sel  : in  std_logic_vector(31 downto 0);
6     pred_out  : out std_logic_vector(31 downto 0);
7     pred_valid : out std_logic
8   );
9 end component;
```

- l'utilisation temporaire d'un *Integrated Logic Analyzer* (ILA) pour le débogage matériel :

Listing 7 – Instanciation de l’ILA
dans `forward_debug_top.vhd`

```

1 ila_i : ila_0
2   port map (
3     clk => pl_clk,
4     probe0 => img_sel_s,
5     probe1 => pred_out_s,
6     probe2 => pred_valid_s
7   );

```

L’ILA permet d’observer les signaux internes critiques et de faciliter l’analyse temporelle ainsi que la validation fonctionnelle du CNN sur FPGA. L’ILA est retiré du design final.

- la visualisation du résultat de classification à l’aide des LEDs de la carte FPGA. Le signal `pred_out_s`, représentant la classe prédite (entre 0 et 9), est affiché sur les LEDs 0 à 3, encodées sur 4 bits. La LED RGB (LED5) indique l’état du calcul et la validité du résultat : verte lorsque la classe prédite correspond à la classe attendue, rouge sinon.

Listing 8 – Extrait de `forward_debug_top.vhd`

```

1 if pred_valid_s = '1' then
2   result_reg <= pred_out_s(3 downto 0);
3   if pred_out_s(3 downto 0) = expected_reg then
4     rgb_state <= CORRECT;
5   else
6     rgb_state <= WRONG;
7   end if;
8 end if;

```

3.4 Validation matérielle

L’implémentation matérielle du réseau de neurones convolutionnel (CNN) a été validée sur FPGA à l’aide de Vivado. L’analyse temporelle post-implémentation montre l’importance de la période d’horloge par exemple à 10 ns, certaines contraintes de timing ne sont pas respectées, avec un slack négatif d’environ -2.3 ns sur le chemin critique.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): -2.333 ns	Worst Hold Slack (WHS): 0.072 ns	Worst Pulse Width Slack (WPWS): 3.500 ns

FIGURE 6 – Analyse temporelle post-implémentation sous Vivado

Les contraintes de hold et de largeur d’impulsion restent néanmoins satisfaites. Le problème a été corrigé en augmentant la période d’horloge, per-

mettant d'obtenir des marges positives pour l'ensemble des contraintes et d'assurer la validité temporelle du design.

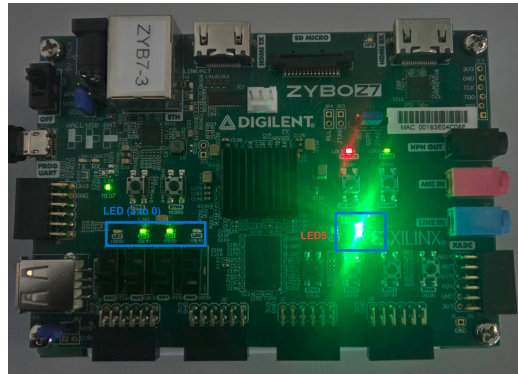


FIGURE 7 – Image 0 et classe prédite 6

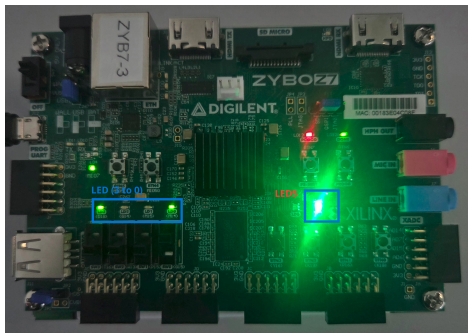


FIGURE 8 – Image 1 et classe prédite 9

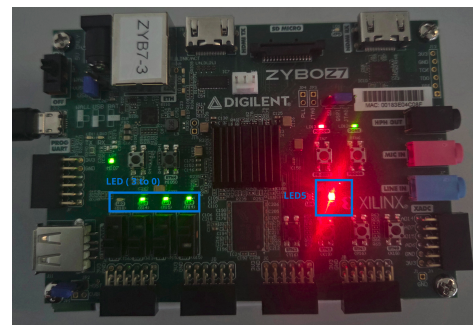


FIGURE 9 – Image 2 et classe prédite 7

FIGURE 10 – Résultats de la classification sur FPGA

Sur FPGA, la classification est quasi instantanée à l'échelle de l'utilisateur. Les prédictions pour les images 0 et 1 sont conformes aux résultats obtenus en Python (Figure 11). Pour l'image 2, le modèle Python produisait déjà une erreur ; la différence observée sur FPGA reste cohérente avec les limites du modèle et de sa mise en œuvre matérielle.

4 Conclusion générale

Ce projet a permis de mettre en œuvre un flot complet de conception d'un réseau de neurones convolutif, depuis un modèle fonctionnel de référence en

Python jusqu'à une implémentation matérielle sur FPGA. L'approche descendante adoptée a assuré une continuité stricte entre les différentes étapes, en garantissant la traçabilité des résultats et la maîtrise des choix d'architecture.

Le modèle Python a servi de référence numérique pour valider l'architecture du CNN et évaluer ses performances sur la base CIFAR-10. La transposition progressive vers une implémentation en virgule fixe HLS a ensuite permis d'identifier les contraintes liées à la synthèse matérielle, tout en conservant une conformité fonctionnelle avec le modèle initial.

Enfin, l'intégration sur FPGA à l'aide de Catapult C et Vivado a démontré la faisabilité d'un CNN entièrement autonome, exécuté en matériel et piloté via des interfaces simples. Les résultats obtenus confirment la pertinence de l'approche HLS pour le prototypage rapide de systèmes d'inférence embarqués et constituent une base solide pour des optimisations futures, notamment en termes de parallélisme, de partitionnement mémoire et de performances.

5 Annexes

```
xph3sei705@cimeld21:~/Documents/ArchicheCNN/CNN_python$ p
ython3 evaluate_CNN_fast.py
Evaluating file: data_batch_1.bin
Image 00000 | True: 6 (frog) | Pred: 6 (frog)
Image 00001 | True: 9 (truck) | Pred: 9 (truck)
Image 00002 | True: 9 (truck) | Pred: 4 (deer)
Image 00003 | True: 4 (deer) | Pred: 4 (deer)
Image 00004 | True: 1 (automobile) | Pred: 1 (automobile)
Image 00005 | True: 1 (automobile) | Pred: 1 (automobile)
Image 00006 | True: 2 (bird) | Pred: 2 (bird)
Image 00007 | True: 7 (horse) | Pred: 7 (horse)
Image 00008 | True: 8 (ship) | Pred: 8 (ship)
Image 00009 | True: 3 (cat) | Pred: 3 (cat)
Accuracy for data_batch_1.bin: 90.00%
=====
Global accuracy on 10 images = 90.00%
=====
```

FIGURE 11 – Python

```
xph3sei705@cimeld21:~/Documents/ArchicheCNN/all_c++
/build$ ./TOP_BENCH
Image 0: True label = 6, Predicted label = 6
Image 1: True label = 9, Predicted label = 9
Image 2: True label = 9, Predicted label = 4
Image 3: True label = 4, Predicted label = 4
Image 4: True label = 1, Predicted label = 1
Image 5: True label = 1, Predicted label = 1
Image 6: True label = 2, Predicted label = 2
Image 7: True label = 7, Predicted label = 7
Image 8: True label = 8, Predicted label = 8
Image 9: True label = 3, Predicted label = 3
=====
Accuracy on 10 images = 90%
=====
```

FIGURE 12 – C++

FIGURE 13 – Equivalence Python/C++ du CNN (10 images)